

Introducción a Spring Boot

IF0009 Desarrollo de software IV

MSI. Álvaro Mena Monge

II-2026



UNIVERSIDAD DE
COSTA RICA

SR-CIE

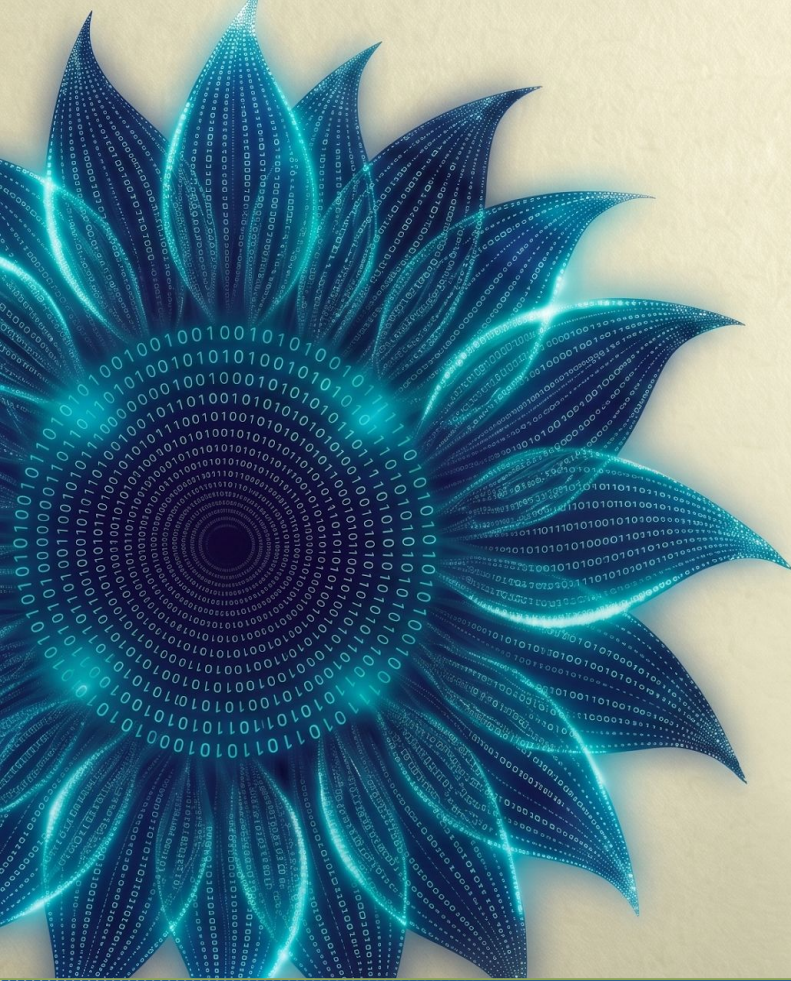
Carrera de
Informática Empresarial
Sedes regionales



Agenda

- El concepto de framework
- El proyecto de “Spring Boot”
- Creación y configuración de la base de datos
- Creación y configuración de un proyecto de aplicación web de *Spring Boot*
- Codificación de una funcionalidad en Spring Boot
 - Codificación de la capa de datos: JDBC
 - Implementación de un controlador MVC @Controller
 - Implementación de la vista con Thymeleaf
- Codificación de un controlador REST @RestController





FRAMEWORK



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

El concepto de framework

Es una pieza de software estructural a partir de la cual parte o aplicaciones de software completas pueden ser organizadas y construidas

- Representan bloques de construcción y de diseño reutilizables
- En el ámbito de la orientación a objetos, un framework consiste de ***clases abstractas y concretas***.
 - La ***instanciación*** de un framework se realiza a través de la ***composición y extensión*** de las clases existentes.



Marcos de construcción: Frameworks

Características

- Automatiza tareas tediosas del dominio
- Introduce una solución arquitectónica elegante
- Proveen procesos claros y componentes predefinidos

Ejemplos Populares

Hibernate

Spring Boot

Apache Struts

JavaServer Faces

EJB

ASP.NET MVC

Angular

Los frameworks actúan como el esqueleto sobre el cual se construye la lógica de negocio específica.



Marcos de construcción: Beneficios

Beneficios



Reutilización de código

Aprovecha componentes probados para acelerar la entrega.



Favorece la productividad

Elimina la necesidad de escribir código repetitivo desde cero.



Estandariza el proceso

Garantiza consistencia y mantenibilidad en todo el ciclo de vida.



Web Application Framework (WAF)

- Web Application Framework (WAF)
 - pieza de software estructural **semi-completa**, **esquelética** y **reusable** que puede ser especializada para producir aplicaciones web a la medida.
 - Usualmente implementan el patrón de diseño **Model-View-Controller** (MVC)



Application Frameworks

- Escoger un framework de aplicación no debe ser una decisión casual
 - ASP.NET MVC Core, JSP Model 2, JSF, Hibernate, Struts 2, Rails, Spring MVC



Ecosistemas de los marcos de trabajo

Decisión Estratégica



Selección Crítica

No debe ser una decisión casual. Impacta la arquitectura, el rendimiento y la mantenibilidad a largo plazo.



Ecosistema de Opciones

Frameworks líderes en la industria:

Spring MVC

ASP.NET Core

Hibernate

Struts 2

Rails

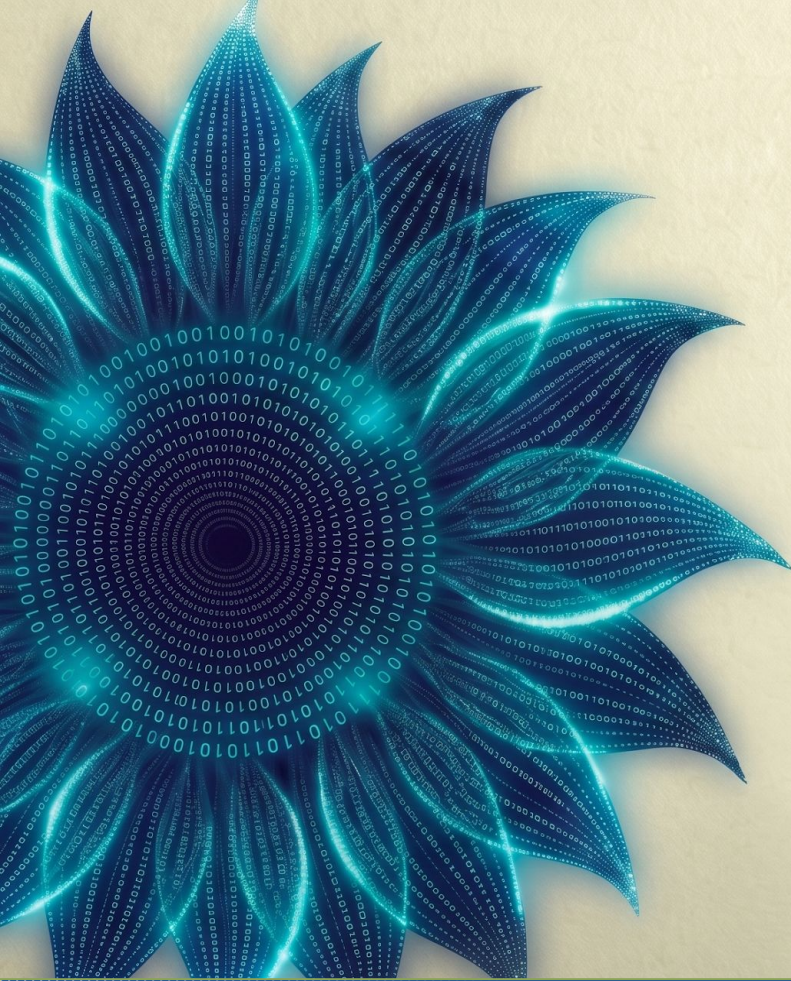
JSF / JSP



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales



El Framework Spring boot



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

Spring Boot

- Creado por Pivotal y liberado en Abril del 2014
- Es un *framework preconfigurado* para el desarrollo de *aplicaciones web*, listas para *producción* de forma muy fácil
- Requerimientos de sistema para Spring Boot 3.4.3

Tomcat 10.1 (10.1.25 or later)

Maven 3.6.3 or later

Java 17 or above





Creación y configuración de la base de datos del curso



Conexión al DBMS del curso

- Abrir el Microsoft Sql Server Management Studio
 - Conectarse al servidor del curso
 - Server name:
163.178.173.130
 - Usuario: Lenguajes
 - Clave: lenguajesparaíso2025

Connect to Server

SQL Server

Login | Connection Properties | Always Encrypted | Additional Connection Parameters

Server

Server type: Database Engine

Server name: 163.178.173.130

Authentication: SQL Server Authentication

Login: Lenguajes

Password: *****

☒ Remember password

Connection Security

Encryption: Optional

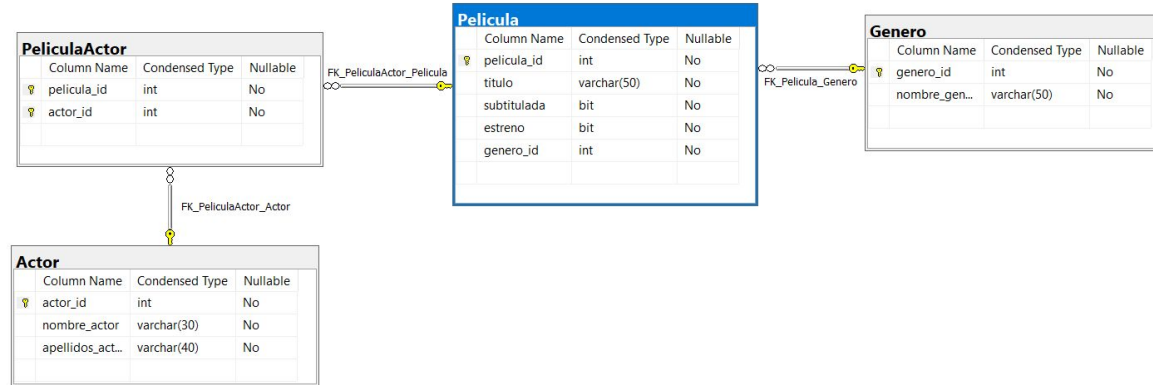
☐ Trust server certificate

Host name in certificate:

Connect Cancel Help Options <<

Entidad-relación b.d. VideoRent

- Reglas de negocio
 - Una película se categoriza en un género
 - Una película está asociada con uno o más autores
- Entidad relación de la b.d. VideoRent



Creación de la BD VideoRent en MS SqlServer

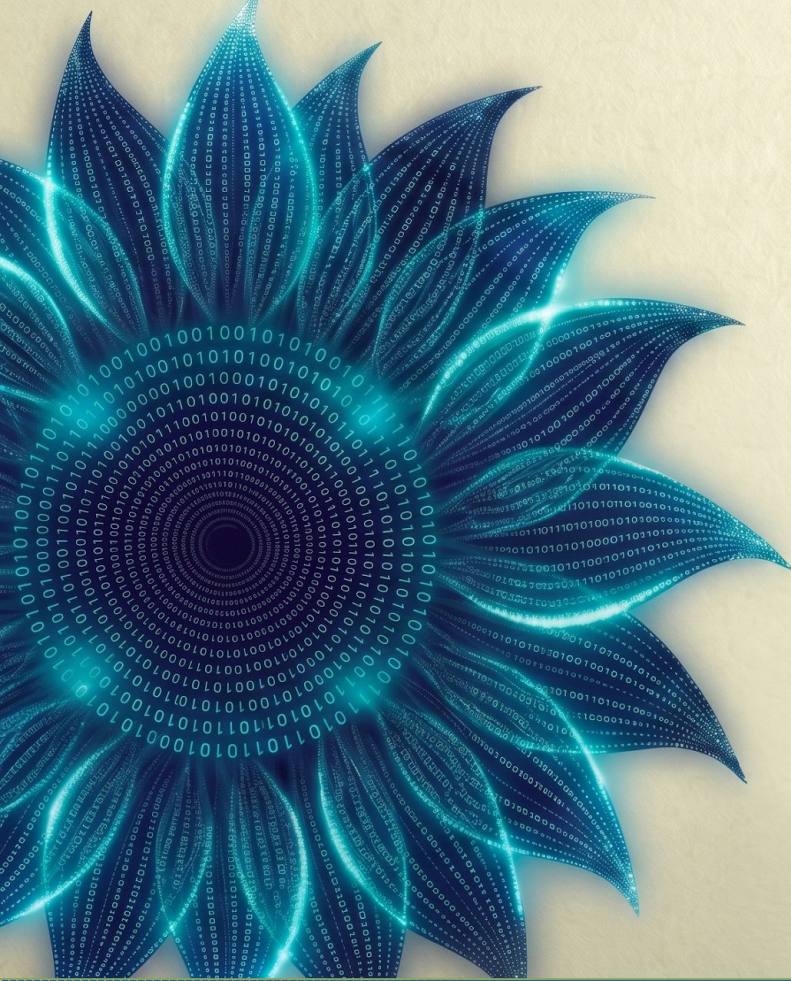
- Creación de la base de datos del curso
 - Nombre: **VideoRent[Carné]_II2026**
- Defina las relaciones de llaves foráneas entre las tablas
 - Pelicula / Genero
 - PeliculaActor / Actor
 - PeliculaActor / Pelicula



Creación de la BD VideoRent en MS SqlServer

- Cree los registros relacionados para su película favorita
 - Incluya 1 película, n autores y 1 género
- Importe los datos desde `registros_videorent.sql`
- Configuración de las llaves primarias como tipo identidad
 - Actor, Pelicula, Genero





Creación de un proyecto en Spring Boot 3.4.3



Configuración de Spring Boot en Visual Studio Code

- . Ctrl+Shift+X para ver el Marketplace
- Instale Extension Pack for Java de Microsoft
- Instale Spring Boot Extension Pack de vmware



Creación de un proyecto de Spring Boot: VSC(1)

Abra la Paleta de Comandos (Ctrl+Shift+P)

Escriba y selecciona: **Spring Initializr: Create a Maven Project**

Complete los siguientes campos:

- Spring Boot version: **4.0.3**
- Language: **Java**
- Group id: **com.videocartago**
- Artifact id: **renting**
- Package:
com.videocartago.renting
- Package type: **jar**
- Java version: **25**



Creación de un proyecto de Spring Boot: VSC(2)

Indique las dependencias:

- MS SQL Server Driver
- Spring Boot DevTools
- JDBC API SQL
- Spring web

Configuración del Runtime

Abra la Paleta de Comandos (Ctrl+Shift+P)

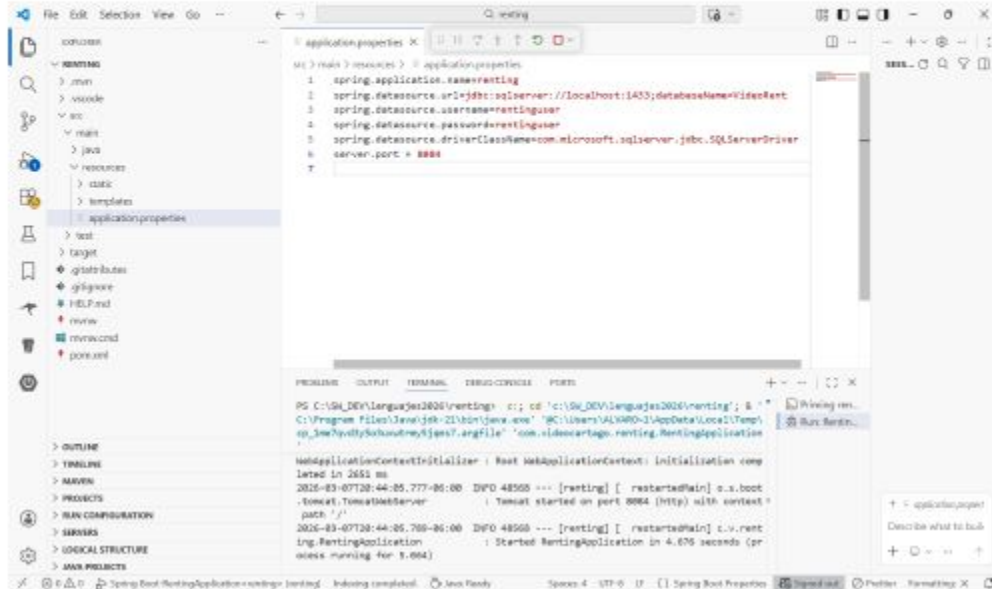
Digite **Java: Configure Java Runtime**

Seleccione la versión correcta 21, 25 en classpath y compiler



Creación de un proyecto de Spring Boot: VSC(3)

Al finalizar la creación del proyecto, se le va a presentar una pantalla como esta. Ubicado en el proyecto, efectúe lo que se le indica:



```
spring.application.name=renting
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=VideoRent
spring.datasource.username=rentinguser
spring.datasource.password=rentinguser
spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver
server.port=8084
```

```
WebApplicationContextInitializer : Root WebApplicationContext: Initialization completed in 2655 ms
2025-03-07T20:44:05.777-06:00 D:\P0 48568 --- [renting] [ restartedMain] o.s.boot
TomcatWebServer : Tomcat started on port 8084 (http) with context
path: /
2025-03-07T20:44:05.789-06:00 D:\P0 48568 --- [renting] [ restartedMain] c.v.rent
ing.RentingApplication : Started RentingApplication in 4.678 seconds (pr
ocess running for 9.064)
```

Agregar la configuración de la fuente de datos

- En el archivo de application properties

```
spring.application.name=videorent
```

```
# URL ajustada con parámetros de seguridad
```

```
spring.datasource.url=jdbc:sqlserver://163.178.173.130;databaseName=[DB NAME];encrypt=true;trustServerCertificate=true;
```

```
spring.datasource.username=Lenguajes
```

```
spring.datasource.password=lenguajesparaizo2025
```

```
spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver
```

```
#context path
```

```
server.servlet.context-path=/renting
```

```
server . port =8084
```



Clase de punto de entrada de la aplicación

- Las aplicaciones Spring Boot debe tener una clase de punto de entrada con el método *main* y la anotación *@SpringBootApplication* para arrancar la aplicación
- La clase debe estar en el paquete raíz.

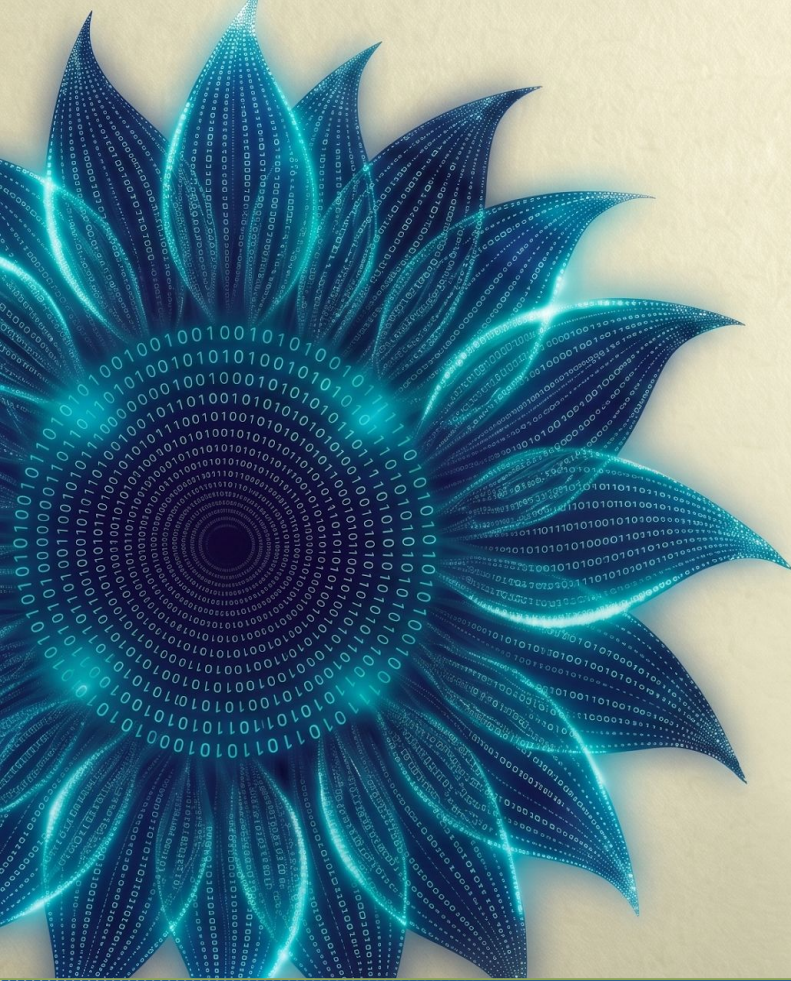
```
package cr.ac.ucr.paraíso.videorent;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class VideorentApplication {

    public static void main(String[] args) {
        SpringApplication.run(VideorentApplication.class, args);
    }
}
```





Thymeleaf



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

ThymeLeaf

- Thymeleaf is a Java library. It is an XML/XHTML/HTML5 template engine able to apply a set of transformations to template files in order to display data and/or text produced by your applications.
- The main goal of Thymeleaf is to provide an elegant and well-formed way of creating templates.
- **Standard Expression syntax**
 - `${...}` : Variable expressions
 - `*{...}` : Selection expressions
 - `#{...}` : Message (i18n) expressions
 - `@{...}` : Link (URL) expressions
 - `~{...}` : Fragment expressions



Thymeleaf

- En el archivo de pom.xml agregar la siguiente dependencia

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-thymeleaf</artifactId>  
</dependency>
```





Creación de una “plantilla”



Adición de un “controller”

```
package cr.ac.ucr.paraíso.videorent.controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.stereotype.Controller;

@Controller
public class WelcomeController {
    @RequestMapping(value="/", method=RequestMethod.GET)
    public String welcome(){
        return "welcome"; // corresponde al nombre de la plantilla
    }
}
```



Adición del "template" → view

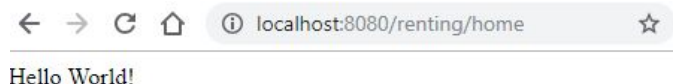
- Crear el html **welcome.html** en la carpeta template de resources

```
HomeController.java helloWorld.html
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="ISO-8859-1">
5 <title>Insert title here</title>
6 </head>
7 <body>
8 Bienvenidos y bienvenidas!!!
9 </body>
10 </html>
```



Correr la aplicación

- Presione el clic derecho del mouse sobre el proyecto
- Seleccione Run AS Spring Boot App
- En el navegador, accese el recurso creado





Creación de un controlador REST



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

Clase tipo RestController

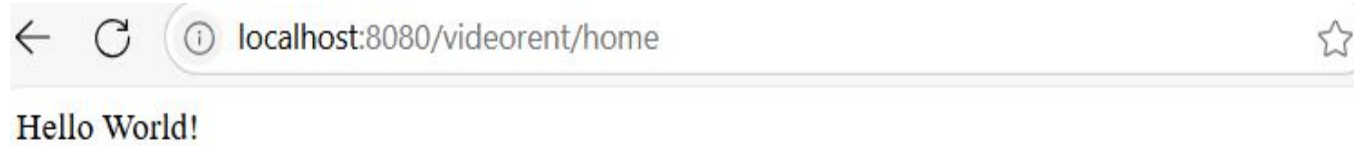
La anotación `@RequestMapping` provee la información de “ruteo” y le dice a Spring que cualquier solicitud HTTP con la ruta `/home` debería mapearse al método “home”

```
package cr.ac.ucr.paraíso.videorent.restcontroller;
import
org.springframework.web.bind.annotation.RequestMapping;
import
org.springframework.web.bind.annotation.RequestMethod;
import
org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloRestController {
    @RequestMapping(value="/home",
method=RequestMethod.GET)
    public String home(){
        return "Hello World!";
    }
}
```



Clase tipo RestController





Let's code!!!
CASO DE ESTUDIO



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

Funcionalidad para “Listar las películas existentes”

← → ↻ 🏠 ⓘ localhost:8080/renting/findMovies 🔍 ☆ 📄 🗑️ 📄

Mantenimiento de películas

Título de la película:

Nombre género:

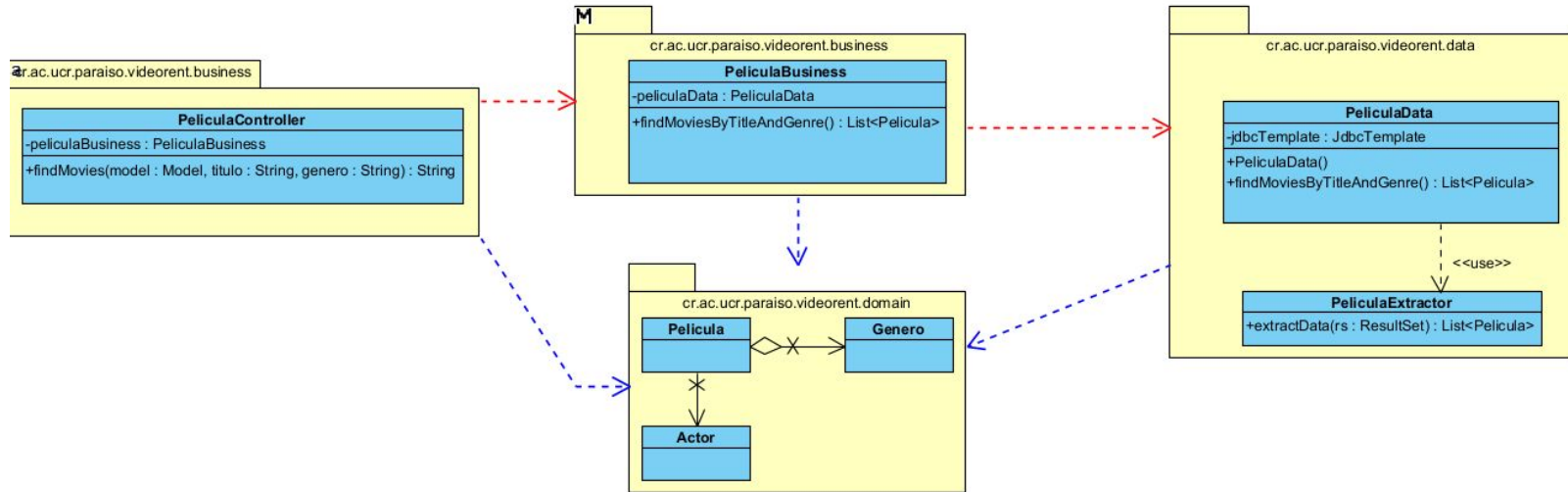
Buscar

Código	Título	Género	Total Películas	Índice
97	Transformers the future	Acción	10000	1
100	X-Men: The Beginning	Acción	10000	2
104	The amazing spiderman	Acción	12	3
58	The Mask of Zorro	Acción	25	4
59	The Net	Acción	20	5

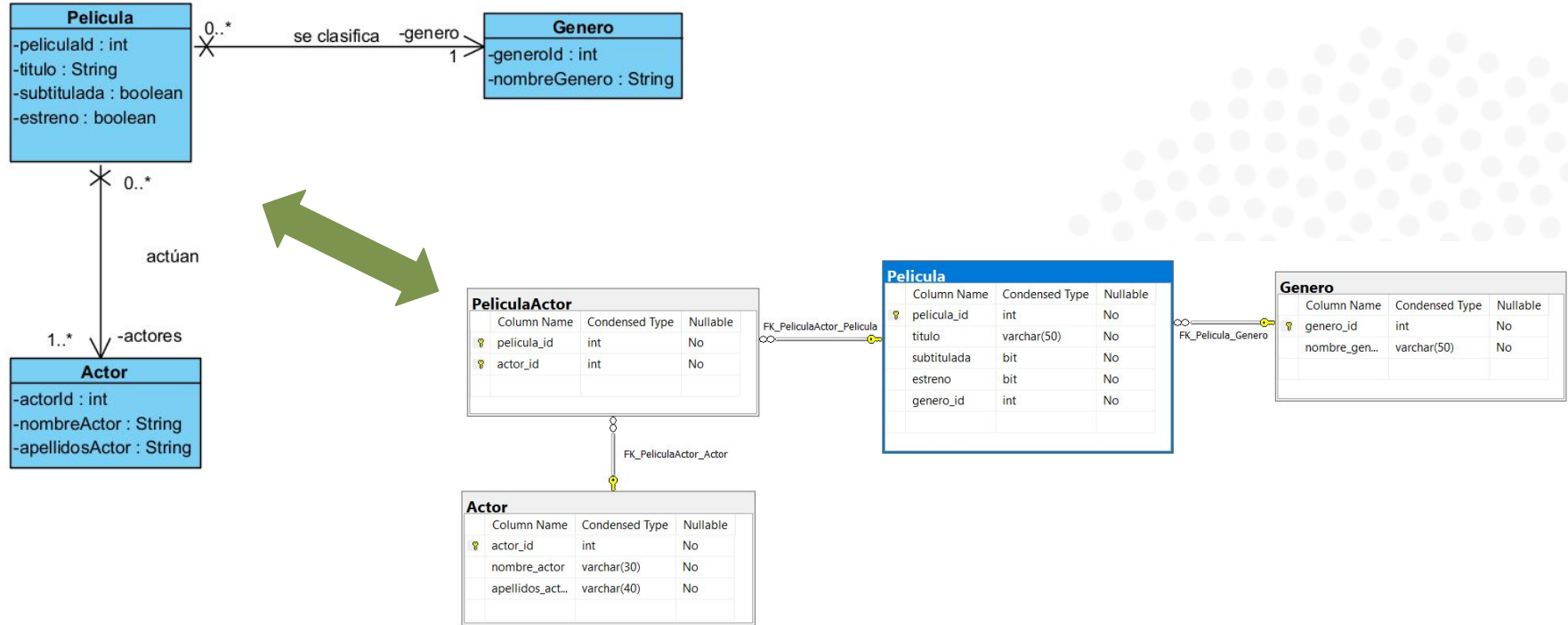


1. Capas y paquetes de la aplicación

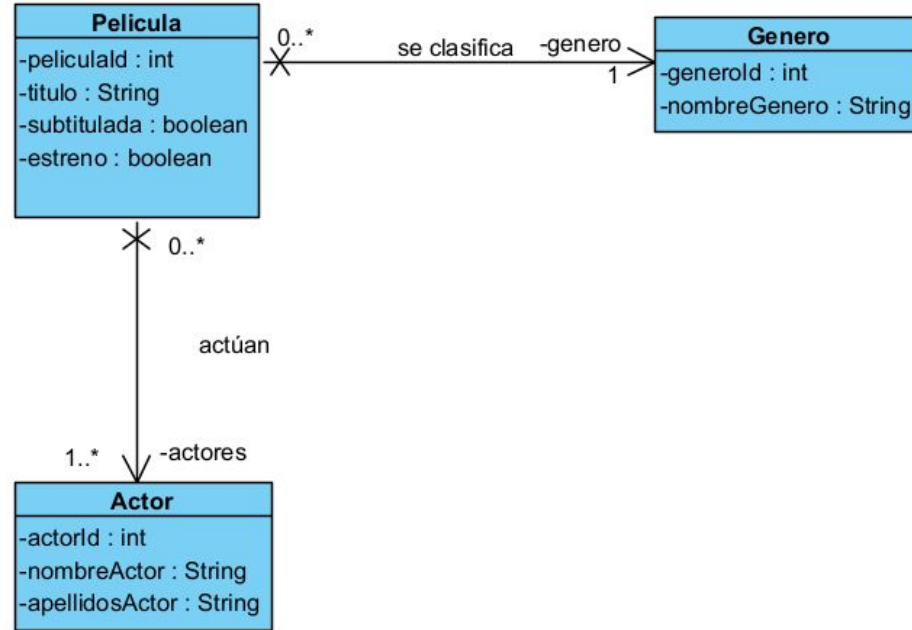
- Paquetes requeridos: data, business, controller, domain



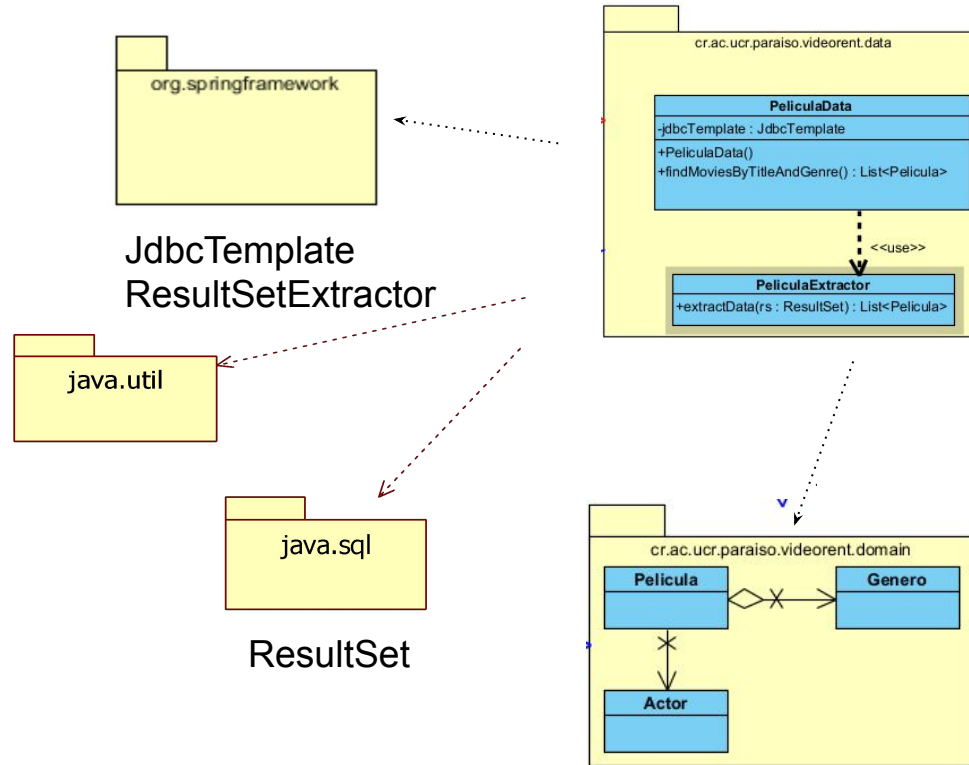
2. Mapeo Objeto-Relacional: el modelo de dominio

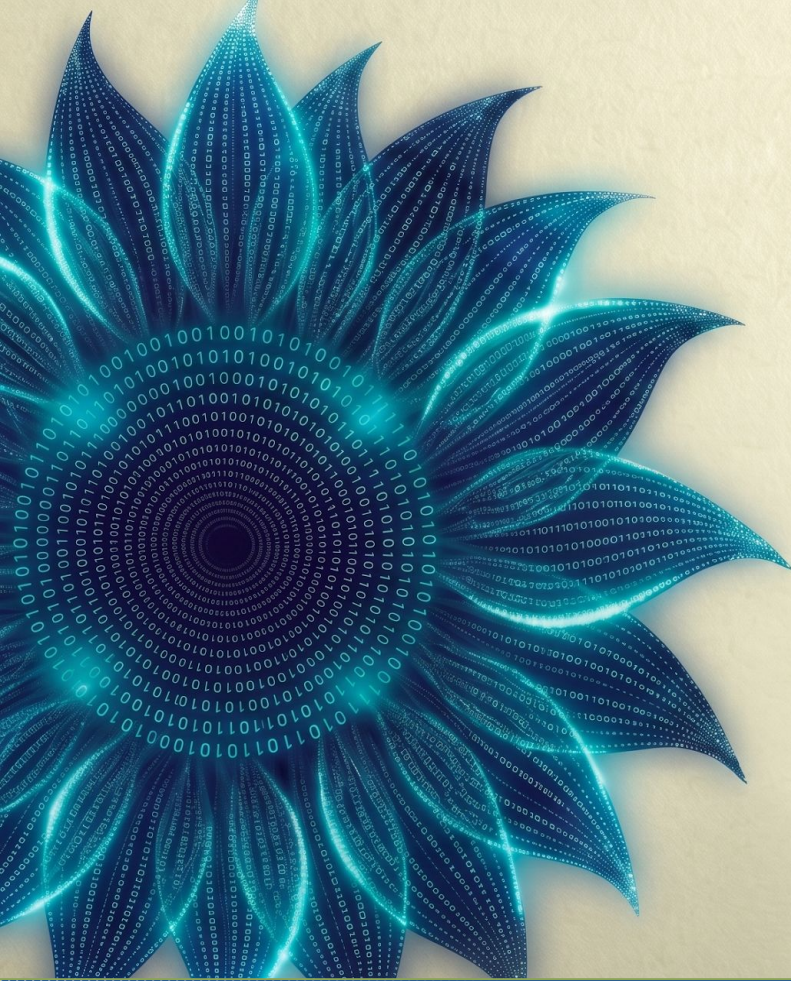


3. Codificación de las clases del dominio



4. Capa de datos (a): Diseño





Capa de datos



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

4. Capa de datos (b): Codificación

```
@Repository
public class PeliculaData {
    @Autowired
    private JdbcTemplate jdbcTemplate;
    @Autowired
    private DataSource dataSource;

    public List<Pelicula> findMoviesByTitleOrGenre(String title, String genre) {
        String sqlSelect = """
            SELECT
                p.pelicula_id,
                p.titulo,
                p.genero_id,
                g.nombre_genero,
                p.subtitulada,
                p.estreno,
                pa.actor_id,
                a.nombre_actor,
                a.apellidos_actor
            FROM Pelicula p
            INNER JOIN Genero g
                ON p.genero_id = g.genero_id
            LEFT JOIN PeliculaActor pa
                ON p.pelicula_id = pa.pelicula_id
            LEFT JOIN Actor a
                ON pa.actor_id = a.actor_id
            WHERE LOWER(p.titulo) LIKE ?
            or LOWER(g.nombre_genero) LIKE ?
            """;
        title = title.toLowerCase();
        genre = genre.toLowerCase();
        String titleLike = (title == null || title.isEmpty()) ? "" : "%" + title.trim() + "%";
        String genreLike = (genre == null || genre.isEmpty()) ? "" : "%" + genre.trim() + "%";
        // Pasamos el SQL, la instancia del extractor y los parámetros
        return jdbcTemplate.query(
            sqlSelect,
            new PeliculaExtractor(),
            titleLike,
            genreLike
        );
    }
}
```



4. Capa de datos (c): Codificación

```
class PeliculasWithActoresExtractor implements ResultSetExtractor<List<Pelicula>> {
    @Override
    public List<Pelicula> extractData(ResultSet rs) throws SQLException, DataAccessException {
        Map<Integer, Pelicula> map = new HashMap<Integer, Pelicula>();
        Pelicula pelicula = null;
        while (rs.next()) {
            Integer id = rs.getInt("cod_pelicula");
            pelicula = map.get(id);
            if (pelicula == null) { // new pelicula record
                pelicula = new Pelicula();
                pelicula.setCodPelicula(id);
                pelicula.setTitulo(rs.getString("titulo"));
                pelicula.getGenero().setCodGenero(rs.getInt("cod_genero"));
                pelicula.getGenero().setNombreGenero(rs.getString("nombre_genero"));
                pelicula.setTotalPeliculas(rs.getInt("total_peliculas"));
                pelicula.setSubtitulada(rs.getBoolean("subtitulada"));
                pelicula.setEstreno(rs.getBoolean("estreno"));
                map.put(id, pelicula);
            } // if
            int actorId = rs.getInt("cod_actor");
            if (actorId > 0) {
                Actor actor = new Actor();
                actor.setCodActor(actorId);
                actor.setNombreActor(rs.getString("nombre_actor"));
                actor.setApellidosActor(rs.getString("apellidos_actor"));
                pelicula.getActores().add(actor);
            } // if
            // TODO Auto-generated method stub
        } // while
        return new ArrayList<Pelicula>(map.values());
    } // extractData
}
```

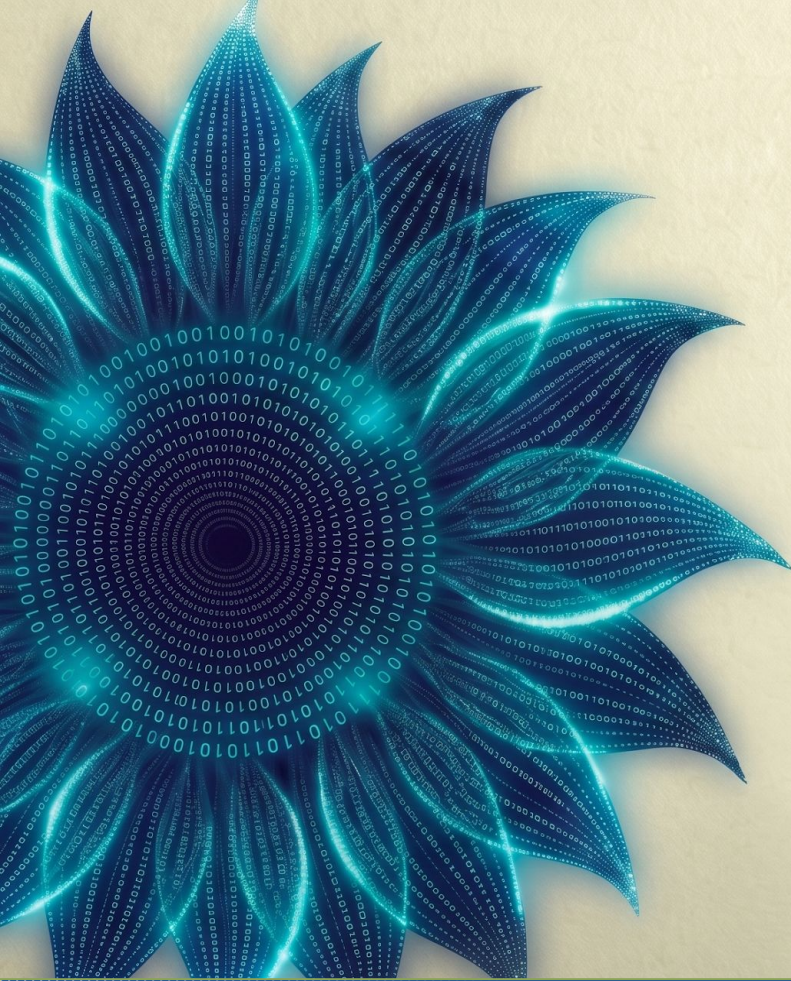


Spring Framework JDBC

¿Qué hace por nosotros?

Action	Spring	You
Define connection parameters.		X
Open the connection.	X	
Specify the SQL statement.		X
Declare parameters and provide parameter values		X
Prepare and execute the statement.	X	
Set up the loop to iterate through the results (if any).	X	
Do the work for each iteration.		X
Process any exception.	X	
Handle transactions.	X	
Close the connection, statement and resultset.	X	





Prueba unitaria



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

4. Capa de datos (d): Prueba unitaria

```
@SpringBootTest
public class PeliculaDataTest {
    @Autowired
    private PeliculaData peliculaData;
    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Test
    @DisplayName("Debe retornar la(s) película(s) cuando el título y el género existen en la base de datos")
    @Transactional // Para que los datos se borren automáticamente al terminar el test
    @Sql(scripts = "/insert_peliculas_con_actores.sql", executionPhase = ExecutionPhase.BEFORE_TEST_METHOD)
    public void givenExistingMovieAndGenre_whenFindMoviesByTitleOrGenre_thenReturnsMatchingMovies() {
        // Arrange
        String title = "Women";
        String genre = "Drama";

        // Act
        var peliculas = peliculaData.findMoviesByTitleOrGenre(title, genre); // MUT method under test
        // Assert

        String expectedTitle = "Women";
        String expectedGenre = "Drama";
        assertNotNull(peliculas);
        assertTrue(!peliculas.isEmpty());

        assertTrue(peliculas.stream().anyMatch(p -> p.getTitulo().contains(expectedTitle) ||
            p.getGenero().getNombreGenero().contains(expectedGenre)));
    }
}
```



4. Capa de negocios (d): Prueba unitaria

```
import static org.junit.jupiter.api.Assertions.*;

import java.util.LinkedList;
import java.util.List;

import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.springframework.boot.test.context.SpringBootTest;

import cr.ac.ucr.paraíso.videorent.data.PeliculaData;
import cr.ac.ucr.paraíso.videorent.domain.Pelicula;

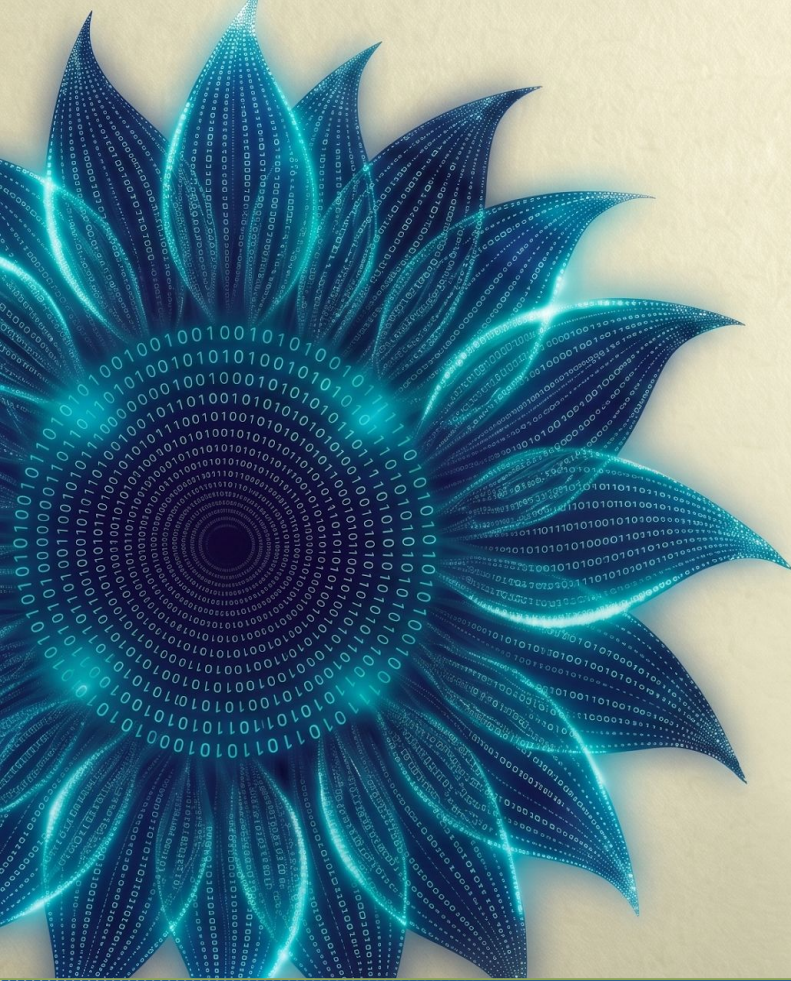
@SpringBootTest
class PeliculaBusinessTest {
    @Mock
    private PeliculaData peliculaData;

    @InjectMocks
    private PeliculaBusiness peliculaBusiness;

    @Test
    void findAllMoviesByTitleAndGenre_cuando_tituloGenero_noEncontrado() {
        // Arrange
        String titulo = "women xyz";
        String genero = "action xyz";
        List<Pelicula> peliculas = new LinkedList<>();
        Mockito.when(peliculaData.findMoviesByTitleAndGenre(titulo, genero)).thenReturn(peliculas);

        //Act
        int valorEsperado = 0;
        List<Pelicula> peliculasRetornadas =
            peliculaBusiness.findAllMoviesByTitleAndGenre(titulo, genero);
        //Assert
        assertEquals(valorEsperado,peliculas.size());
    }
}
```





Capa de la vista



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

5. Capa de aplicación (a)

- Mostrar películas en una forma web
 - El binomio Controller-View
 - Mostrar películas utilizando ThymeLeaf y CSS

Controller	View
PeliculaController	verPeliculas.html

ThymeLeaf

← → ↻ 🏠 ⓘ localhost:8080/renting/findMovies 🔍 ☆ 🚫 👤

Mantenimiento de películas

Título de la película:

Nombre género:

Código	Título	Género	Total Películas	Índice
97	Transformers the future	Acción	10000	1
100	X-Men: The Begining	Acción	10000	2
104	The amazing spiderman	Acción	12	3
58	The Mask of Zorro	Acción	25	4
59	The Net	Acción	20	5

5. Capa de aplicación (b): Controller

```
1 package cr.ac.ucr.paraíso.videorent.controller;|
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
6 import org.springframework.web.bind.annotation.RequestMapping;
7 import org.springframework.web.bind.annotation.RequestMethod;
8 import org.springframework.web.bind.annotation.RequestParam;
9
10 import cr.ac.ucr.paraíso.videorent.business.PeliculaBusiness;
11
12
13
14 @Controller
15 public class PeliculaController {
16     @Autowired
17     private PeliculaBusiness peliculaBusiness;
18
19     @RequestMapping(value="/findMovies", method=RequestMethod.GET )
20     public String iniciar(Model model) {
21         return "findMovies";
22     }
23
24     @RequestMapping(value="/findMovies", method=RequestMethod.POST )
25     public String findMovies(Model model, @RequestParam("titulo") String titulo,
26         @RequestParam("genero") String genero) {
27         model.addAttribute("peliculas", peliculaBusiness.findAllMoviesByTitleAndGenre(titulo, genero));
28         return "findMovies";
29     }
30 }
31
```



6. Capa de presentación : View

localhost:8080/renting/findMovies

Mantenimiento de películas

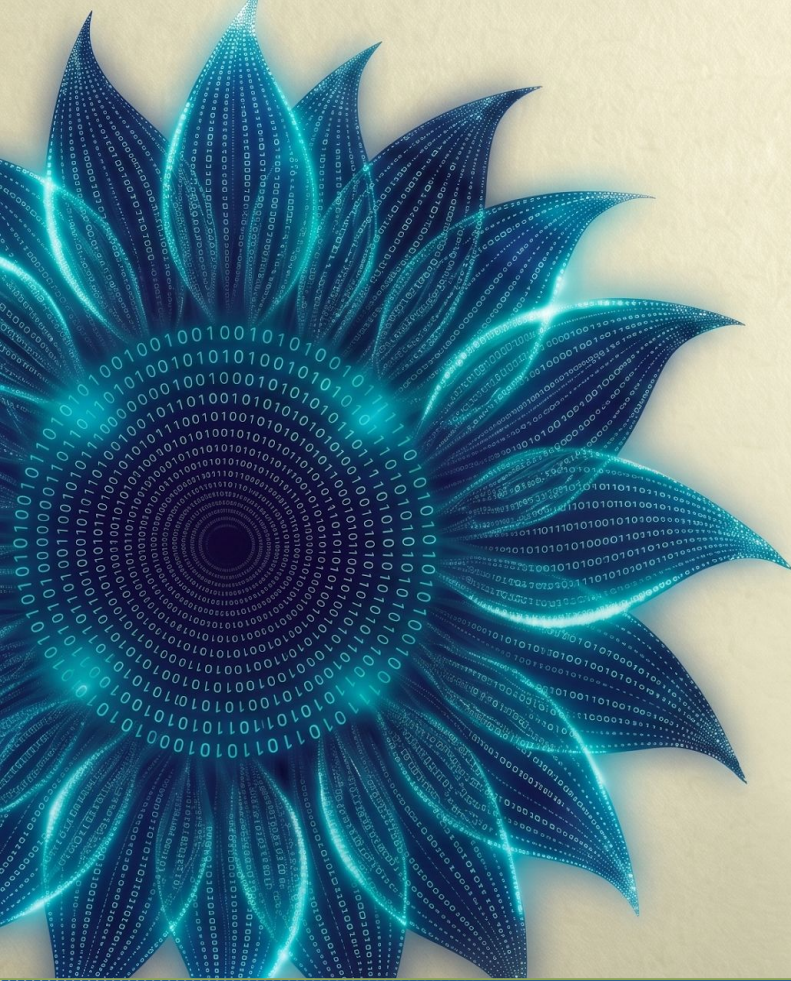
Título de la película:

Nombre género:

Código	Título	Género	Total Películas	Índice
97	Transformers the future	Acción	10000	1
100	X-Men: The Begining	Acción	10000	2
104	The amazing spiderman	Acción	12	3
58	The Mask of Zorro	Acción	25	4
59	The Net	Acción	20	5

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Mantenimiento de películas</title>
6 </head>
7 <body>
8   <div class="container">
9     <h1>Mantenimiento de películas</h1>
10    <form action="#" th:action="@{/findMovies}"
11      method="post">
12      Título de la película: <input type="text" name="titulo" /> <br>
13      Nombre género: <input type="text" name="genero" />
14      <input type="submit" value="Buscar" />
15    </form><br>
16    <table>
17      <tr>
18        <th>Código</th>
19        <th>Título</th>
20        <th>Género</th>
21        <th>¿Subtitulada?</th>
22        <th>Índice</th>
23      </tr>
24      <tr th:each="pelicula : ${peliculas}">
25        <td><span th:text="${pelicula.peliculaId}"></span></td>
26        <td><span th:text="${pelicula.titulo}"></span></td>
27        <td><span th:text="${pelicula.genero.nombreGenero}"></span></td>
28        <td><span th:text="${pelicula.subtitulada}"></span></td>
29        <td><span th:text="${peliculaStat.count}"></span></td>
30      </tr>
31    </table>
32  </div>
33 </body>
34 </html>
```





Controlador REST



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes regionales

6. Controlador REST:

```
@RestController
@RequestMapping(value = "/peliculas")
@CrossOrigin(origins = "http://localhost:4200")
public class PeliculaRestController {

    @Autowired
    private PeliculaBusiness peliculaBusiness;

    @Autowired
    private PeliculaMapper peliculaMapper; // Inject the mapper

    @GetMapping
    public ResponseEntity<List<Pelicula>> findMovies(@RequestParam("titulo") String titulo,
        @RequestParam("genero") String genero) {
        List<Pelicula> peliculas = peliculaBusiness.findMoviesByTitleOrGenre(titulo, genero);

        if (peliculas.isEmpty()) {
            return ResponseEntity.noContent().build();
        }

        return ResponseEntity.ok(peliculas);
    }
}
```



Cliente REST para Visual Studio Code

1

Instalar Extensión

Instale la extensión **REST Client** de Huachao Mao en Visual Studio Code.

2

Crear y Ejecutar

Cree un archivo con extensión **.http** (ej. `rest-client.http`) y ejecute la acción **GET**.

```
rest-client.http

@baseUrl = http://localhost:8084/renting
### Search movies by title or genre

GET {{baseUrl}}/peliculas?titulo=Little
Women&genero=Drama

Accept: application/json
Content-Type: application/json
```



Referencias

Data access with JDBC. (n.d.).

<https://docs.spring.io/spring/docs/current/spring-framework-reference/html/jdbc.html>

Getting started with the Standard dialects in 5 minutes (n.d.).

<http://www.thymeleaf.org/doc/articles/standarddialect5minutes.htm>

Prasad, S. (2017). Beginning Spring Boot 2: Applications and Microservices with the Spring Framework. Apress, New York, USA.

Thymeleaf (2024). *The Thymeleaf Interactive Tutorial.*

<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>

